DUST on the small code-carried model: the dust_schedule sweep at 512 draws per bottleneck

2026-10-27

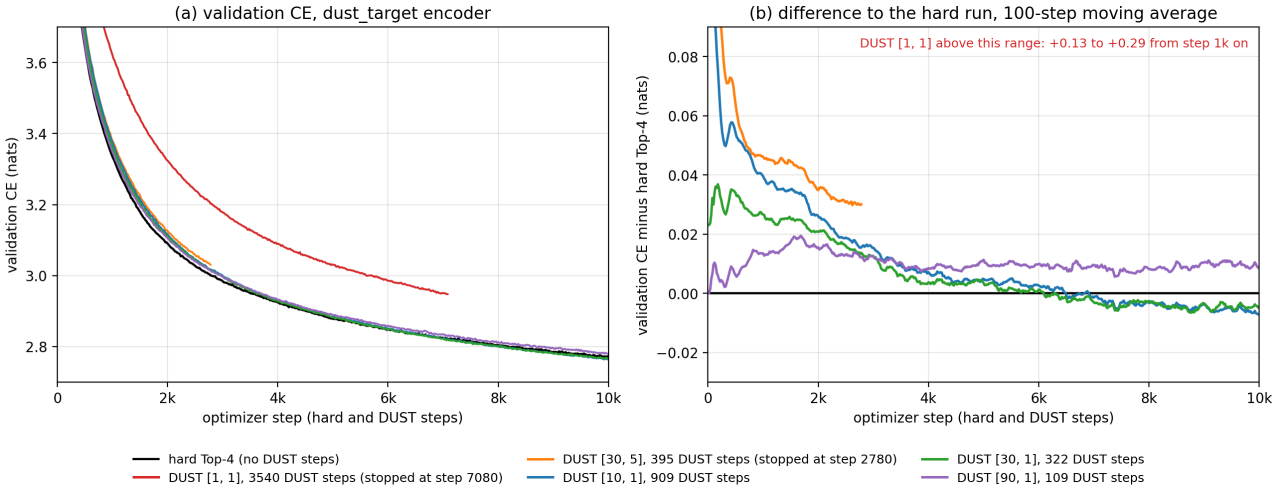


Figure 1: dust_target encoder. Model: 4 blocks of width 128 (2 heads), a Top-4 bottleneck of $N = 512$ features at `residual_out` of every block with the code carried between blocks (`code_residual`), no bottleneck post-norm, magnitude-direction decoupling, the 8192-token BPE of TinyStories, batch 96×512 tokens, constant learning rate $6 \cdot 10^{-4}$, no gradient clipping, 10 000 optimizer steps, seed 1337. The black curve trains with hard Top-4 steps only. The coloured curves interleave DUST steps by `dust_schedule` $[n_h, n_d]$: every period of $n_h + n_d$ optimizer steps starts with n_h hard steps followed by n_d DUST steps; the legend gives the number of DUST steps up to the last plotted step. DUST settings: 512 draws per bottleneck on the whole pool $J = N - K = 508$, $\sigma = 0.1$ (absolute), $\gamma = 0$, a separate Adam instance for the DUST steps at the same learning rate (`dust_lr_scale` 1), the draws split over 8 GPUs. (a) Validation CE: 40 batches of 24 windows of 512 tokens, the same windows in every run, every 20 steps; the axis starts at 3.7 nats, so the first ≈ 500 steps are not shown. (b) Validation CE minus that of the hard run at the same step, centred moving average over 5 validations (100 steps); between neighbouring validations the unsmoothed difference fluctuates by about 0.0015 nats. The $[1, 1]$ run (red) lies above the range of (b). The $[1, 1]$ and $[30, 5]$ runs were stopped by request, at steps 7096 and 2796; their last validations are at steps 7080 and 2780. Single runs.

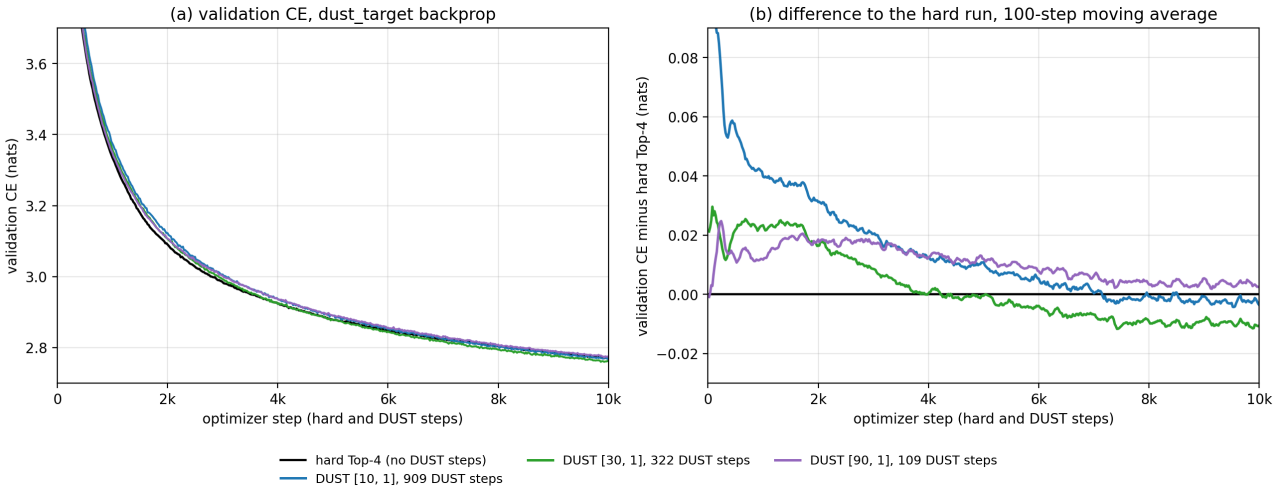


Figure 2: dust_target backprop. As Figure 1, with the DUST estimate at every gate input back-propagated to every parameter before that gate (`dust_target backprop`) instead of applied to the gate's encoder only. No $[1, 1]$ or $[30, 5]$ run with this target.

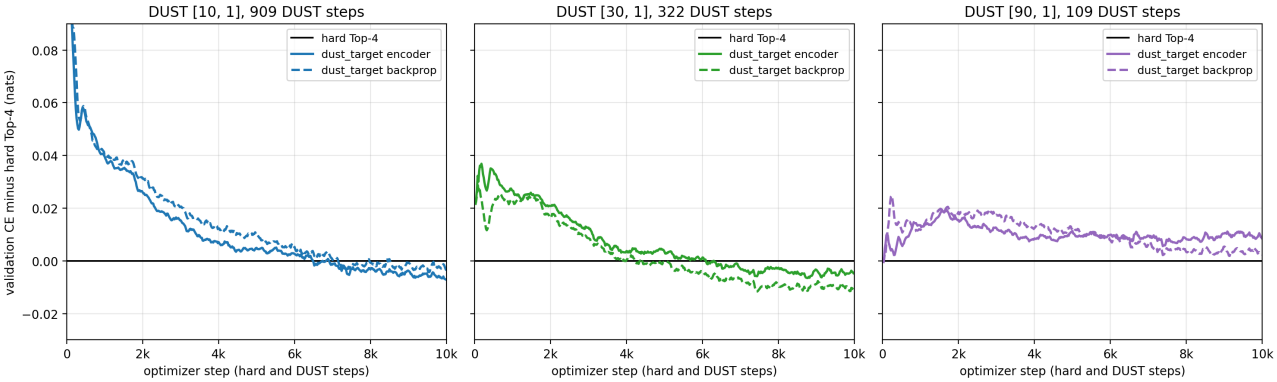


Figure 3: Encoder and backprop targets per schedule. The curves of Figures 1b and 2b, one panel per `dust_schedule`: validation CE minus that of the hard run at the same step, 100-step moving average; solid encoder, dashed backprop.

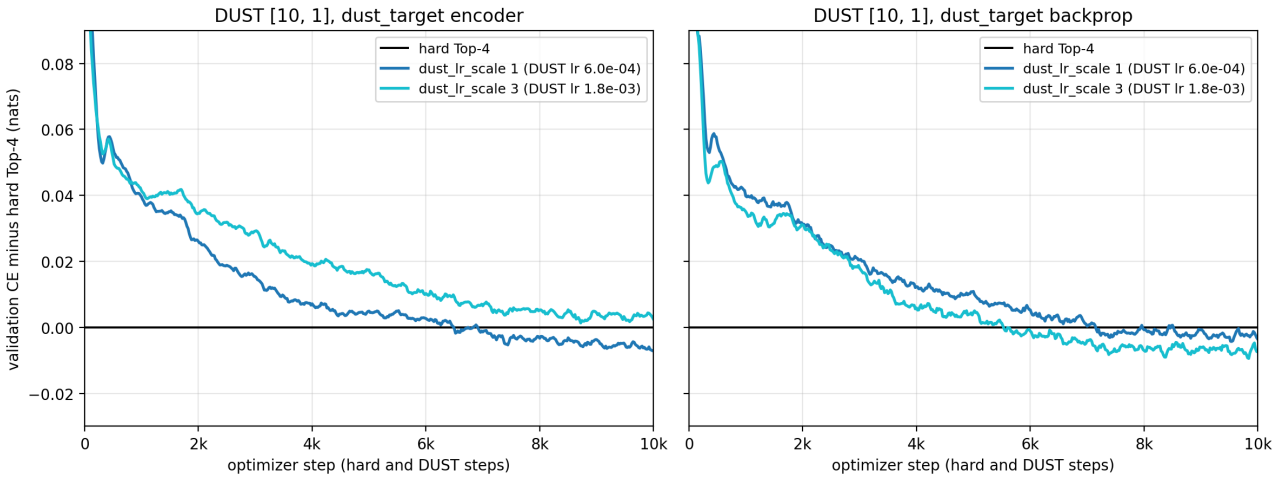


Figure 4: dust_lr_scale at $[10, 1]$**.** The learning rate of the DUST optimizer at $6{\cdot}10^{-4}$ (`dust_lr_scale` 1, the runs of Figures 1–3) and at $1.8 \cdot 10^{-3}$ (`dust_lr_scale` 3); the hard steps use $6 \cdot 10^{-4}$ in all runs. Validation CE minus that of the hard run at the same step, 100-step moving average; left encoder, right backprop.

Runs

Each run is at
`gdrive:weight-sparsity/runs_warsaw/<run>/`
(box warsaw, 8× RTX 5090) with `metrics.jsonl`, `config.yaml`, `config.json`, the TensorBoard events in `tb/`, and a checkpoint every 500 steps (`ckpt_step<N>.pt`, all kept, and `latest.pt`); the stopped runs carry `stopped.json`. The configs are `configs/small_dust/<run>.yaml`; the figures are made by `scripts/plot_small_dust_sweep.py` from the `metrics.jsonl` files.

run	`dust_schedule`	`dust_target`	lr scale	steps	checkpoints
`wa_k4_hard`	(hard only)	–	–	10 000	500–10 000
`wa_k4_dust_p512`	$[1, 1]$	encoder	1	7096*	500–7000
`wa_k4_dust_p512_h30_d5`	$[30, 5]$	encoder	1	2796*	500–2500
`wa_k4_dust_p512_h10`	$[10, 1]$	encoder	1	10 000	500–10 000
`wa_k4_dust_p512_h30`	$[30, 1]$	encoder	1	10 000	500–10 000
`wa_k4_dust_p512_h90`	$[90, 1]$	encoder	1	10 000	500–10 000
`wa_k4_dust_p512_bp_h10`	$[10, 1]$	backprop	1	10 000	500–10 000
`wa_k4_dust_p512_bp_h30`	$[30, 1]$	backprop	1	10 000	500–10 000
`wa_k4_dust_p512_bp_h90`	$[90, 1]$	backprop	1	10 000	500–10 000
`wa_k4_dust_p512_h10_lr3`	$[10, 1]$	encoder	3	10 000	500–10 000
`wa_k4_dust_p512_bp_h10_lr3`	$[10, 1]$	backprop	3	10 000	500–10 000

* stopped by request; lr scale is `dust_lr_scale`; a checkpoint every 500 steps.